CCBDA Project Final Report

Max Eidet[1], Olle Pettersson[1], Benedikt Prisett[1], Patrik Timko[1], and Ilyas Bakchiche[1]

[1]Facultat d'Informàtica de Barcelona - Universitat Politècnica de Catalunya

May 2026

Abstract

This project implements a cloud-native Job Application Assistant that helps users compare their CVs with specific job descriptions. Users can upload CVs and job postings through a Next.js frontend, while a FastAPI backend manages authentication, storage, and analysis workflows using AWS services. Uploaded documents are stored in Amazon S3 and processed through an asynchronous pipeline using AWS Lambda, SQS, Textract, and Bedrock. The system extracts structured candidate profiles and job requirements, then generates match scores, missing skills, and recommendations. DynamoDB stores application data, while Cognito provides secure user authentication and user-specific access control.

The project demonstrates an end-to-end AWS architecture combining serverless processing, managed storage, AI-based document analysis, authentication, monitoring, and scalable deployment. It shows how cloud services can be integrated to build a practical tool that helps job seekers tailor their applications more effectively. The project is publicly available at https://github.com/benediktpri/ccbda-project

1 Introduction

Applying for jobs is often a repetitive and time-consuming process. Candidates must interpret job requirements, compare them with their own experience, and identify which skills or qualifications should be emphasized or improved for each position. When this evaluation is done manually, it can be difficult to maintain consistency across applications, and candidates may overlook important gaps between their profile and the expectations of a specific role.

This project presents the development of a cloud-based Job Application Assistant that supports this evaluation process. Users authenticate through AWS Cognito, upload their CV as a PDF, and provide job descriptions for positions of interest. The system then uses automated document processing and Large

Language Models (LLMs) to extract, structure, and compare information from both the candidate profile and the job description. The result is a personalized job-fit analysis containing a match score, identified matching and missing skills, a natural-language summary of the candidate's suitability for the role, recommendations for skill improvement, and suggestions for how the CV can be better tailored to the specific job description.

The application is implemented as an end-to-end, event-driven cloud solution on AWS. The frontend is built with Next.js and served through Amazon S3 and CloudFront, while the FastAPI backend is deployed on Elastic Beanstalk and acts as the main API gateway. Uploaded documents are stored in Amazon S3 and processed asynchronously through a decoupled serverless pipeline using AWS Lambda, Amazon SQS, Amazon Textract, and Amazon Bedrock. Extracted profiles, structured job descriptions, and analysis results are stored in Amazon DynamoDB, while Amazon CloudWatch is used for logging, monitoring, dashboards, and alarms.

The primary goal of the project is twofold: to create a practically useful tool that helps job seekers evaluate their qualifications against specific roles, and to demonstrate the design and implementation of a realistic, production-like distributed cloud application. By combining managed authentication, asynchronous processing, serverless components, AI-based analysis, and observability, the project applies modern cloud computing principles in a concrete real-world use case.

2 Motivation

The motivation for this project was to apply the knowledge gained throughout the course in a practical and integrated cloud-based system, rather than working only with isolated lab exercises. The project demonstrates key cloud architecture principles such as designing for failure, elasticity, decoupled components, security, and event-driven processing.

Our application builds on core AWS services used during the course, including S3, Elastic Beanstalk, Lambda, SQS, DynamoDB, and CloudWatch, while also extending the solution with Cognito for authentication, Textract for document text extraction, and Bedrock for AI-powered CV and job analysis. The system also follows several twelve-factor principles, such as environment-based configuration, stateless backend processes, and centralized logging.

The use case was chosen because it is directly relevant to us as students preparing to enter the job market. By helping users analyze their CV against job descriptions and identify skill gaps, the application provides a practical and meaningful real-world context. At the same time, it combines synchronous API requests with asynchronous document processing pipelines, making it a suitable project for exploring modern cloud application design beyond a traditional CRUD system.

3 Functionalities

The application is implemented as a web-based platform consisting of a frontend developed using Next.js and a backend powered by FastAPI. It integrates several interconnected features designed to provide an efficient and intuitive user experience throughout the job application support process.

The following subsections describe the main functionality of the system according to the typical user workflow, beginning with profile creation and resume processing, followed by job posting submission and automated analysis, and concluding with personalised feedback and recommendations. The underlying technologies and implementation details of these features are discussed in later sections.

3.1 User Profile

After visiting the platform, the user is presented with a login page where they can either sign in to an existing account or create a new one. During registration, the user is required to provide an email address and password, both of which are validated according to standard security requirements. After submitting the registration form, the user must enter a verification code sent to their email address in order to prevent spam and automated bot registrations.

Following the initial signup, the user is guided through an onboarding process. They may upload their resume, which is automatically processed to extract relevant information such as education, work experience, and skills. This step is optional, as users can alternatively complete the profile manually through a form as shown in the Figure 1

After the resume is processed, the extracted information is pre-filled into the form so the user can review, edit, or supplement the detected data. This ensures both convenience and accuracy of the stored profile information as you can see in the Figure 2

Once the onboarding process is completed, the user gains access to the main application, where they can further view and edit their profile details at any time.

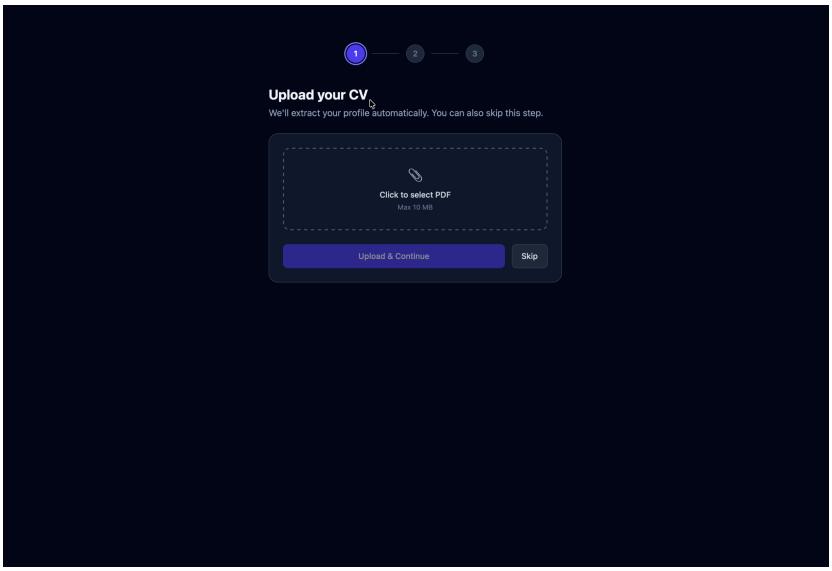

Figure 1: User profile workflow, showing the onboarding or profile page where CV information can be uploaded.

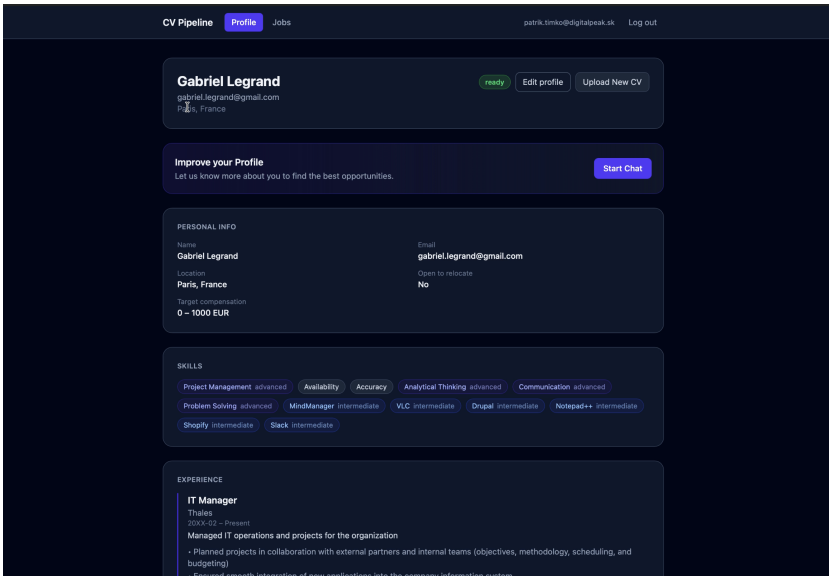

Figure 2: User profile workflow, showing the onboarding or profile page where CV information can be reviewed, and edited..

3.2 Job Postings

Users can access the Job Postings section of the application. Initially, this page is empty, but users can add job postings either by pasting the job description text directly into the application or by uploading a PDF document containing the posting.

After submission, the application extracts relevant information from the job posting, including the position title, company name, required skills, and other important details. The extracted data is then formatted and displayed in a clear and user-friendly interface as shown in the Figure 3

Users can subsequently manually request an analysis comparing their profile against the uploaded job posting and receive a personalised feedback.

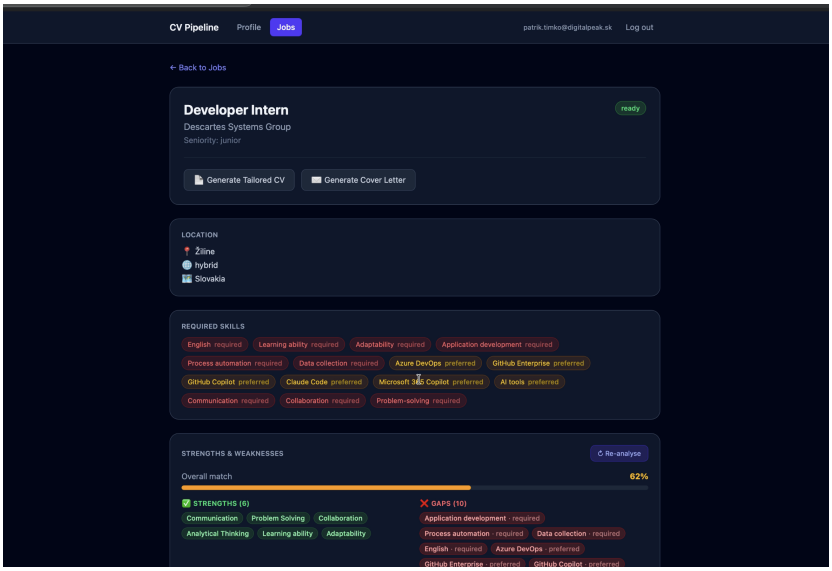

Figure 3: Extraction of a Job application through an upload of a PDF document and informations about it

3.3 Personalised Feedback

After the analysis is completed, the user is presented with an alignment score ranging from 0–100%, representing the estimated suitability of the user for the selected position.

The analysis also provides a detailed overview of matching and missing skills, accompanied by a textual summary containing recommendations and observations regarding the user's strengths and weaknesses in relation to the specific job posting as shown in the Figure 4

Additionally, the system generates personalised suggestions for improving the user's resume in order to increase the likelihood of success during the application process. These recommendations focus on emphasising relevant skills

and experiences while reducing the prominence of unrelated content as you can see in Figure 4

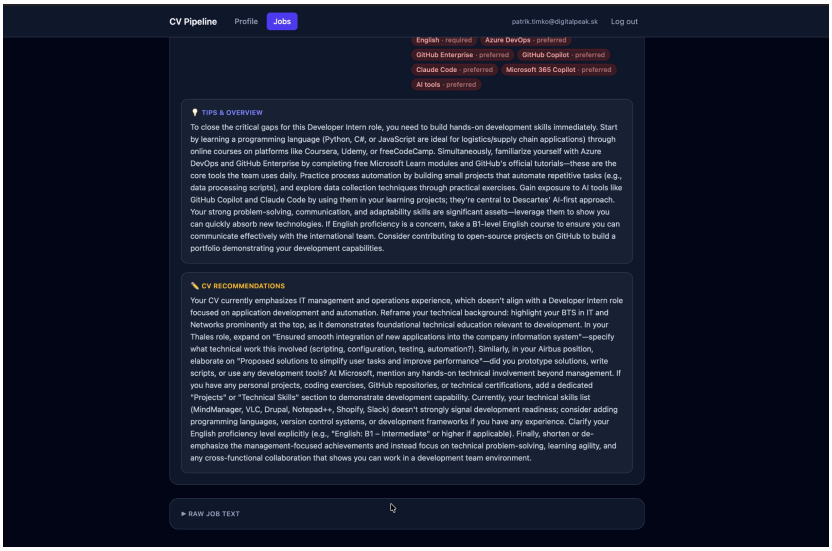

Figure 4: Personalised feedback page, matched and missing skills, and recommendations for improving the application.

4 System Architecture

The Job Application Assistant is built as a cloud-native application on AWS, designed to follow the best practices established in the course. Figure 5 gives an overview of the overall architecture and the interactions between the AWS services involved. At a high level the system is composed of:

- **A static frontend**: a Next.js application served from S3 and distributed via CloudFront, which the user interacts with directly in the browser.
- **A synchronous backend API**: a FastAPI service running in a Docker container on Elastic Beanstalk, exposing all CRUD operations and the final skills-gap analysis.
- **An asynchronous processing pipeline**: a chain of four AWS Lambda functions, connected by S3 events and SQS queues, that handles the long-running work of extracting and structuring text from uploaded PDFs.

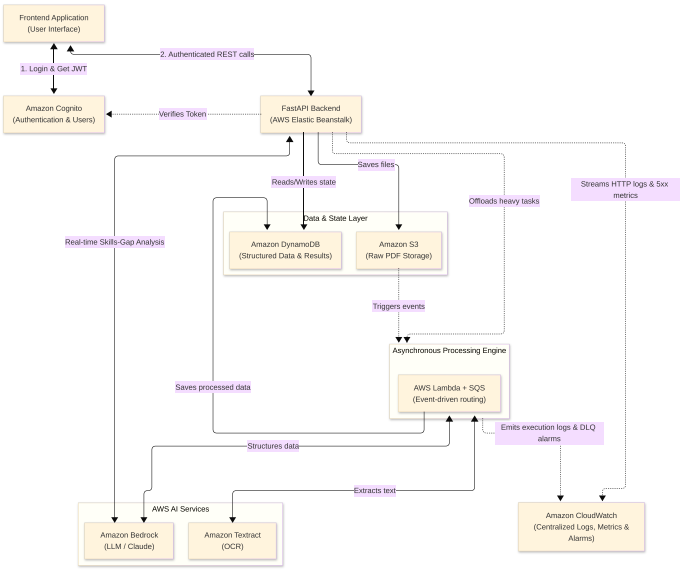

Figure 5: High-level overview of the system architecture, showing the main application components and how the frontend, backend, authentication, storage, document processing, AI analysis, and monitoring services interact.

The FastAPI backend exposes a small set of REST endpoints, organised into five routers: `users`, `upload`, `profiles`, `jobs`, and `results`. Together they cover the full CRUD interaction a user needs: uploading a CV, viewing and editing the extracted profile, managing job descriptions, and triggering the skills-gap analysis. The route functions themselves contain little logic of their own and instead delegate to a service layer: they parse and authenticate the request, then call into `services/dynamodb.py` for storage, an S3 client for file uploads, or an SQS client to enqueue work. Anything that involves Textract or Bedrock is pushed off the request path and handled asynchronously by the document processing pipeline described next. Authentication, further detailed in section 4.4, is delegated entirely to Amazon Cognito: the browser obtains a JWT directly from the Cognito User Pool and attaches it as a bearer token to every subsequent backend request, which the API verifies against Cognito's JWKS endpoint.

The asynchronous document processing pipeline, further detailed in section 4.3, is built with four Lambda functions connected by S3 events and two SQS queues. It extracts text from uploaded PDFs with Amazon Textract, turns that text into structured profile and job data using an LLM hosted on Amazon Bedrock, and writes the results back to DynamoDB.

All application state (users, raw and structured profiles, jobs, and analysis results) is persisted in a single DynamoDB table using a partition-key/sort-key design that keeps every record belonging to a user retrievable in one query (section 4.2). Every component, from the FastAPI middleware to each Lambda,

emits structured JSON logs to Amazon CloudWatch, where dashboards and alarms make the system observable end-to-end (section 4.5). The entire stack is deployed to the `eu-west-1` region from GitHub Actions on tag push (section 4.7).

4.1 Backend API

The backend is a FastAPI application written in Python 3.13, packaged as a Docker image and deployed on AWS Elastic Beanstalk. All routes are mounted under a common `/api` prefix so that the same CloudFront distribution can serve the static frontend from S3 and forward `/api/*` requests to Elastic Beanstalk, giving the browser a single origin and avoiding CORS complications in production.

The application is organised into three layers. The first, `routers/`, defines the HTTP surface, split into the five routers introduced earlier (`users`, `upload`, `profiles`, `jobs`, `results`). The second, `services/`, encapsulates the integrations with AWS: `services/dynamodb.py` centralises all single-table CRUD, `services/auth.py` verifies Cognito JWTs against the cached JWKS, and `services/bedrock.py` performs the synchronous skills-gap analysis call. The third, `models/schemas.py`, defines Pydantic models that are reused for both request/response validation and DynamoDB serialisation, which keeps a single source of truth for the shape of every entity in the system.

A custom HTTP middleware logs every request as a structured JSON record (method, path, status, duration), which is picked up by CloudWatch via the Elastic Beanstalk Docker log driver and feeds the dashboards and alarms described in section 4.5. For local development a `AUTH_BYPASS` flag short-circuits the JWT check so that the API can be exercised without a Cognito session; the flag is hard-coded off in deployed environments.

Unlike the separate document processing pipelines, the skills-gap analysis of a CV against a job is performed synchronously by the backend itself. When the user triggers an analysis on the frontend, the `results` router fetches the user's structured profile and the structured job from DynamoDB, calls `services/bedrock.py`, persists the result, and returns it to the client in the same response.

For the functionalities of the application that require reasoning over text, the skills-gap analysis described above and the profile and job structuring described in the next section, we use Amazon Bedrock as the model provider, with Claude Haiku 4.5 as the underlying model. This model was chosen for its low per-token cost and fast response times, which proved more than sufficient for the relatively simple tasks the system asks of it.

An alternative would have been to use the OpenAI or Anthropic APIs directly, as they offer mature, well-documented APIs. However, we chose AWS Bedrock, which allowed us to work with and learn about another AWS service that fits well into our overall architecture. Another benefit is that Bedrock exposes a single interface across multiple model families (Claude, Llama, Mistral, etc.), which makes it easy to swap the underlying model without touching

application code.

One trade-off of using Bedrock is that it does not currently offer a native structured-output mode of the kind exposed by OpenAI or Anthropic's own APIs. Since all of our functionalities depend on the model returning JSON that conforms to a fixed schema, we work around this by leaning on Claude's tool-use feature: the desired schema is passed as a tool definition and the model is forced to invoke that tool, which guarantees a schema-conformant response and removes any need for prompt-parsing on the application side.

4.2 Data Storage and Persistence

The application has two distinct kinds of data to persist: the binary source documents that users upload (CV and job-description PDFs), and the structured application state derived from them (users, raw and structured profiles, jobs, and analysis results). These two kinds of data have very different access patterns, so they are kept in two different services: Amazon S3 for the files and Amazon DynamoDB for everything else.

File storage on S3. All uploaded PDFs are stored in a single S3 bucket (`ccbda-app-bucket`), partitioned by document type and user using key prefixes: CVs are written under `profiles/{user_id}/{uuid}.pdf` and job descriptions under `jobs/{user_id}/{job_id}.pdf`. This layout has two practical benefits. First, it scopes every object to the user that uploaded it, which keeps the IAM and application-level authorisation logic simple. Second, it gives the asynchronous pipeline a cheap way to decide what kind of document it is processing: the same Text Extractor Lambda is wired to S3 `ObjectCreated` events for the whole bucket, and inspects the key prefix to route the extracted text to either the profile or the job SQS queue (see section 4.3). S3 is a natural fit for this part of the system because the files are immutable, can be large, and are read sequentially by Textract rather than queried. Storing them in a database, by contrast, would have been both more expensive and operationally awkward.

Application state on DynamoDB. All structured application data is stored in a single DynamoDB table called `AppTable`, configured with on-demand (pay-per-request) billing. The table follows the single-table design pattern commonly used for DynamoDB: rather than creating a separate table per entity, every record type lives in the same table and is distinguished by the combination of its partition key (`PK`) and sort key (`SK`).

Because every entity in the system belongs to exactly one user, we use the user identifier as the partition key (`PK = USER#{user_id}`) and encode the entity type and identifier in the sort key. Concretely, a user's partition contains the rows shown in Table 1.

Entity	SK	Notable fields
User metadata	`METADATA`	`created_at`
Raw profile	`PROFILE#RAW`	`status, raw_text, s3_key`
Structured profile	`PROFILE#STRUCTURED`	`skills[], experience[], education[], languages[]`
Raw job	`JOB_RAW#{job_id}`	`status, source_type, raw_text`
Structured job	`JOB#{job_id}`	`title, company, required_skills[]`
Analysis result	`ANALYSIS#{job_id}`	`match_score, matched/missing_skills[], recommendations`

Table 1: Single-table layout for `AppTable`. All rows share the same partition key `USER#{user_id}`; the sort key encodes the entity type and, where relevant, the entity identifier.

This layout was chosen so that the application's access patterns map directly to a single DynamoDB `Query` on the partition key, with optional `begins_with` conditions on the sort key.

Why DynamoDB and not a relational database. The obvious alternative for the application state was a relational database, specifically Amazon RDS with PostgreSQL. We chose DynamoDB instead for three reasons. First, the access patterns are narrow and known up front: every read and write is scoped to a single user, which removes the main reason one would reach for SQL — flexible, ad-hoc joins across entities. Second, the cost profile fits a course project: DynamoDB on-demand has no instance to size and is effectively free when idle, whereas even the smallest RDS instance runs continuously and would dominate the AWS bill. Third, the data is naturally semi-structured (nested lists of skills, experiences, education), which would have meant either several joined tables or a JSON column in PostgreSQL, giving up some of the relational benefits anyway. The cost of this decision is that we hand-roll the few "joins" we do need on the application side, and that adding a fundamentally new access pattern may later require a GSI or a key re-shape.

4.3 Document Processing Pipelines

The application uses two document-parsing pipelines one for CVs and one for job descriptions. Both pipelines follows the same general design: the FastAPI backend handles the user-facing request, while heavier processing tasks are moved to asynchronous AWS services. This keeps the API responsive and allows document extraction and LLM processing to happen independently of the frontend request.

When a user uploads a CV, the FastAPI backend stores the PDF in Amazon S3 using a user specific key. At the same time, a raw profile item is created

in DynamoDB with status `pending`, which allows the frontend to track that proccessing has started.

The S3 upload triggers the Text Extractor Lambda through an `ObjectCreated` event. This lambda downloads the PDF and sends it to Amazon Textract for OCR, and extracts the raw textlines from the response. After this extraction, the lambda routes the extracted CV text to CV SQS queue. The Profile Structurer Lambda consumes messages from this queue and uses Amazon Bedrock to transform the unstructured CV text into a structured candidate profile containing skills, experience, education, languages and other relevant fields. The result is stored in DynamoDB with status `ready`. This CV processing flow can be seen in Figure 6.

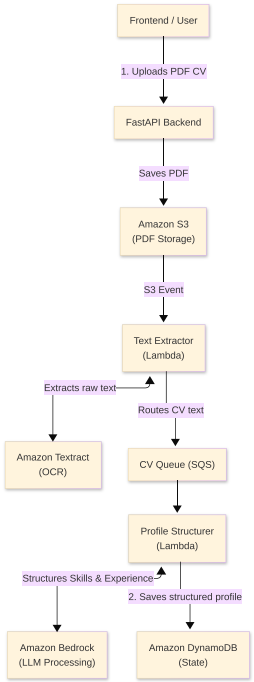

Figure 6: CV processing pipeline from PDF upload to structured profile storage.

The job description processing pipeline follows a very similar architecture. However, unlike CV uploads, job descriptions can enter the system either as uploaded PDF documents or as directly pasted text. This introduces a small variation in the processing flow depending on whether OCR is required. If the user uploads a PDF, the backend stores it in S3 under a `jobs/` prefix. This triggers the same Text Extractor Lambda, which identifies the document type from the S3 key and routes the extracted text to the job-processing queue.

If the user pastes the job description as plain text, OCR is not needed. In that case, the FastAPI backend sends the text directly to the job SQS

queue. This avoids unnecessary S3 and Textract processing. The Job Structurer Lambda consumes messages from the job queue and uses Bedrock to extract structured job information, such as title, company, seniority, required skills, responsibilities, and qualifications. The structured job is then saved in DynamoDB. Once both the structured profile and structured job are available, the user can request a skills-gap analysis, which, as previously described, is handled by the FastAPI backend. This job-processing flow can be seen in Figure 7.

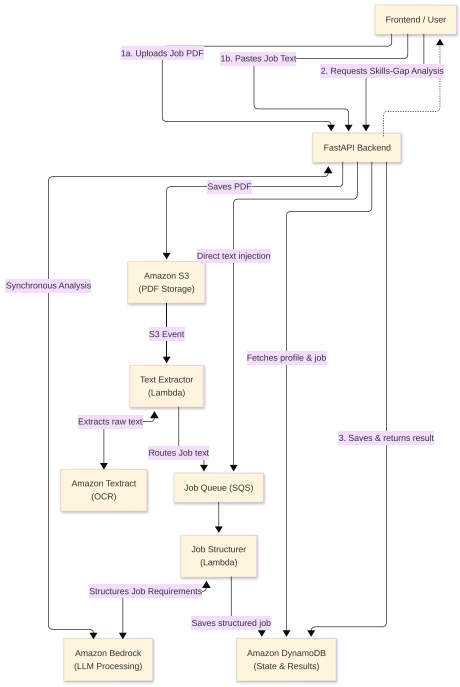

Figure 7: Job description processing pipeline and skills-gap analysis flow.

Design Choice AWS Lambda, Amazon SQS, and Amazon Textract were chosen because they fit the event-driven nature of the document-processing workflow. Uploading a CV or job description is not simple database operation. The document must be stored, parsed, transformed into text and later interpreted by an LLM. Running all of this synchronously inside the FastAPI request would make the API slower, less reliable, and more sensitive to timeouts. Lambda allows the processing steps to run only when needed. This keeps the backend lightweight and lets the processing pipeline scale independently from the user-facing API. Textract was used because it is a managed OCR service for extracting text from PDF documents, which avoids building and maintaining our own parsing or OCR solution. This is important because CVs and job descriptions can have different layouts and formatting. SQS was used to decouple the extraction step from the structuring step. After Textract extracts raw text,

the result is sent to a queue instead of directly invoking the next processing component. This makes the system more fault tolerant, if the Bedrock-based structuring Lambda fails or is temporarily throttled, the message can remain in the queue and be retried. It also helps absorb spikes in uploads without overloading downstream services. Together, Lambda, SQS, and Textract create a scalable and resilient pipeline for document processing.

4.4 Authentication and Security

AWS Cognito is employed as the central Identity Provider (IdP) to establish a secure identity baseline. A native AWS solution was chosen to align directly with the project's scope and deployment architecture.

Cognito Configuration and Architecture The authentication infrastructure utilizes two core components:

- **AWS Cognito User Pool:** Functions as the managed user directory, securely storing credentials, enforcing strict complexity policies, and managing automated email verification codes.
- **Cognito App Client:** Acts as the public cryptographic gateway for the browser. It is explicitly configured without a *client secret* to mitigate token exposure risks within client-side JavaScript, leveraging standard OAuth 2.0 public flows with verified emails as the primary login identifier.

User Registration and Verification Flow The user onboarding and authentication lifecycle transitions accounts from registration to an active local session, as illustrated in Figure 8.

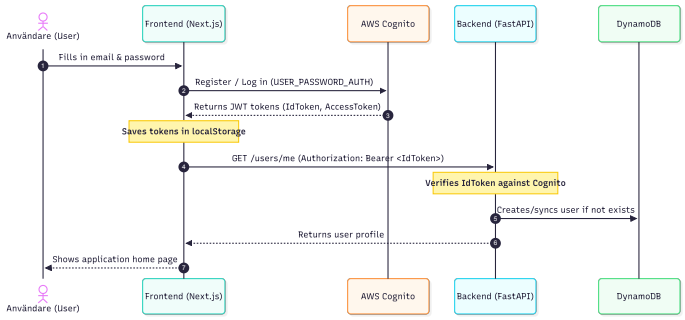

Figure 8: User flow for Cognito registration, verification, and authentication.

The workflow executes across five distinct phases:

1. **Sign-up:** The user registers via the frontend client, provisioning an initially unconfirmed user record in the Cognito User Pool.

2. **Verification:** The user submits the short-lived numeric verification code dispatched to their email, transitioning the account status to confirmed.

3. **Sign-in:** The frontend authenticates credentials directly via a secure `USER_PASSWORD_AUTH` flow, receiving a cryptographic trio of tokens (Identity, Access, and Refresh) stored locally in browser `localStorage`.

4. **Identity Mapping:** The frontend decodes the `IdToken` to extract the immutable subject identifier (`sub`), adopting it as the application-level `user_id`.

5. **Synchronization:** The client dispatches a request to the backend's `GET /users/me` endpoint with the token. The FastAPI backend verifies the identity and retrieves or provisions the corresponding user row in Amazon DynamoDB.

Runtime Token Validation and Authorization The FastAPI backend implements a zero-trust model where protected endpoints require a JWT Bearer token in the HTTP `Authorization` header. Processing is managed by a custom `CognitoAuthenticator` layer:

1. **Cryptographic Validation:** The backend caches Cognito's public JSON Web Key Sets (JWKS) to verify the token's signature, expiration (`exp`), issuer (`iss`), and audience (`aud`).

2. **Authorization Enforcement:** The verified `sub` claim is extracted via FastAPI dependency injection (`get_current_user`). A secondary dependency (`verify_user_id`) cross-references this ID against the requested path parameter (e.g., `/users/{user_id}`). Any mismatch short-circuits the request with an HTTP `403 Forbidden` exception to prevent cross-user resource spoofing.

Data Isolation and Infrastructure Integration The system enforces a strict separation of concerns across infrastructure layers:

- **Identity Data:** User credentials, verification states, and security policies are isolated exclusively within AWS Cognito.

- **Application Data:** User metadata and application states are isolated within Amazon DynamoDB utilizing a single-table design, where the verified Cognito `sub` is adopted as the primary partition key structured as `PK = USER#`$\langle sub\_id \rangle$.

Current Limitations and Future Work The current security baseline has several limitations designated for future iterations:

- **Token Storage:** Transitioning token storage from browser `localStorage` to secure, server-side `HttpOnly` cookies to eliminate vulnerability to client-side scripts.

- **Session Lifecycle:** Integrating server-side session invalidation via Cognito's `GlobalSignOut` API and automating token renewal utilizing the cached `RefreshToken`.
- **User Management:** Implementing critical self-service identity routines, including password recovery workflows and verification code resending capabilities.

4.5 Observability/ Logging

Observability in our system is implemented mainly through Amazon CloudWatch Logs, CloudWatch Metrics, alarms, and dashboards. Since the application consists of several distributed AWS components, including Elastic Beanstalk, Lambda, SQS, DynamoDB, Textract, and Bedrock, centralized logging is necessary to trace execution across service boundaries. This is especially important for the asynchronous document-processing pipeline, where a single user action can trigger an S3 upload, a Lambda invocation, Textract extraction, SQS message delivery, Bedrock processing, and DynamoDB updates.

The FastAPI backend uses a centralized logging module, implemented in `logger.py`, that configures the Python root logger to emit structured JSON logs to standard output. Each log entry includes fields such as `timestamp`, `level`, `logger`, and `message`, and exception stack traces are included when available. The backend also contains HTTP middleware that logs every API request with additional structured fields, including `http_method`, `http_path`, `http_status`, and `duration_ms`. This makes it possible to query backend behaviour in CloudWatch Logs Insights, for example by filtering slow requests, identifying repeated 5xx responses, or inspecting failed user operations.

In Elastic Beanstalk, the application logs are streamed from the Docker container output to CloudWatch Logs. This follows the Twelve-Factor approach where the application does not manage log files itself, but writes logs as an event stream to stdout. Elastic Beanstalk is then responsible for forwarding those logs to CloudWatch. In our CloudWatch setup, log retention is configured and application logs can be inspected without SSH access to the underlying EC2 instance.

The Lambda functions use the same structured logging approach. Each handler configures the Lambda logger to output single-line JSON records, which are automatically collected in CloudWatch log groups under `/aws/lambda/<function-name>`. The Lambda logs record important state transitions in the pipeline, such as receiving an S3 event, extracting text with Textract, sending a message to SQS, invoking Bedrock, writing structured data to DynamoDB, and marking failed items from the dead-letter queue. This gives us an execution trace for each stage of the pipeline and makes it easier to locate where a failed or delayed document processing job stopped.

CloudWatch is also used for operational monitoring through metrics, alarms, and dashboards. The deployment script creates an SNS topic for alarm notifications and connects the CloudWatch alarms to this topic. This means that alarms

can trigger email notifications once a subscriber has been added and confirmed. The script configures alarms for Lambda errors, messages visible in dead-letter queues, FastAPI 5xx responses from Elastic Beanstalk, unusually high Lambda invocation volume, and high Bedrock invocation volume. These alarms cover both reliability issues and cost-related risks, since repeated Lambda or Bedrock invocations could indicate a bug, retry loop, or abuse scenario.

The CloudWatch dashboard combines service metrics and log queries in one place, including Lambda invocations, Lambda errors, SQS queue depth, DLQ queue depth, Bedrock invocations, Bedrock input/output token usage, and recent application logs. The dashboard also filters out low-value runtime noise such as Lambda `START`, `END`, and `REPORT` messages, making the displayed logs more focused on application behaviour.

Design Choice We chose CloudWatch because it integrates directly with the AWS services used in the project. Lambda, Elastic Beanstalk, SQS and Bedrock already publish logs or metrics to CloudWatch, which means we can get useful observability without deploying and maintaining a separate logging platform. This fits the project goal of using managed cloud services and keeps the architecture simpler. CloudWatch also supports the Twelve-Factor principle of treating logs as event streams, since the application writes logs to standard output and the platform collects them centrally. Alternative solutions were considered. The ELK stack would provide powerful log analysis and visualization, but requires significantly more infrastructure and maintenance. Datadog offers advanced observability features, but introduces external dependencies and higher operational costs. CloudWatch was ultimately chosen because it integrates natively with AWS while providing sufficient logging, metrics, alarms, and dashboards with minimal operational overhead.

4.6 Frontend

The frontend application was developed using the Next.js framework, which was chosen for its modern architecture, efficient client-side navigation, and the team's prior experience with the framework. The project was configured to be exported as a static site, eliminating the need to run a dedicated Next.js server in the deployment environment. For styling, the TailwindCSS library was used due to its simplicity, flexibility, and ability to enable rapid iteration during development.

The frontend serves as the primary interface between the user and the system, providing all user-facing functionality, including authentication, profile management, job posting uploads, and the presentation of analysis results.

The user interface was designed with a strong focus on simplicity and usability. The application follows modern web design conventions to ensure an intuitive and responsive experience across a wide range of devices and screen sizes. A consistent dark mode theme is used throughout the interface to enhance readability and reduce visual strain, particularly during extended use.

Communication between the frontend and backend is handled through a REST API. All interactions are performed asynchronously, allowing the interface to remain responsive during longer-running operations such as document parsing and analysis generation. While real-time communication using WebSockets was initially considered for operations such as resume processing, it was ultimately not implemented due to time constraints. Instead, a polling-based approach was adopted to periodically check the status of long-running tasks.

4.7 Deployment Setup

The whole stack is deployed to AWS from GitHub Actions, with two separate workflows. The CI workflow (`ci.yml`) runs on every push and pull request to `main` and gates merges: it lints and formats the backend with `ruff`, type-checks and builds the frontend, runs the backend test suite (using `moto` to mock AWS), and validates that the Bedrock extraction schemas are still consistent with the Pydantic models. The deploy workflow (`deploy.yml`) is triggered exclusively by pushing a version tag matching `v*`, which keeps deployments explicit and gives us a trivial command for re-deploying after a teardown.

The deploy workflow runs four sequential jobs. `setup-cognito` runs `scripts/setup_cognito.sh`, which creates the User Pool and app client if they do not already exist (and reuses them otherwise), and exports their IDs as job outputs. `deploy-backend` patches `eb-options.json` with those Cognito IDs, builds a Docker source bundle, uploads it to the EB artifact bucket, registers a new application version, and updates the Elastic Beanstalk environment, polling until it reports `Ready`. `deploy-lambdas` packages and deploys the four pipeline Lambdas via `scripts/deploy_pipeline.sh`, including the S3 trigger and the SQS event-source mappings. `deploy-frontend` builds the Next.js app with the Cognito IDs and API URL injected as `NEXT_PUBLIC_*` variables, syncs the static `out/` directory to the frontend S3 bucket, and invalidates the CloudFront distribution.

A guiding principle of the workflow is that every step is idempotent. The Cognito script reuses an existing pool and client by name, the EB job recreates a terminated environment but only updates option settings on a live one, and the pipeline script reuses existing S3 buckets, SQS queues, IAM roles, and Lambda functions. This means the same tag-push command that deploys a fresh environment from scratch also re-deploys a partially torn-down one, without manual cleanup or state surgery.

Stopping idle costs. Our course project sits idle most of the time, but two parts of the stack incur cost or noise even when nobody is using the application. The Elastic Beanstalk environment runs an EC2 instance continuously, which is the dominant idle cost. The four SQS event-source mappings on the pipeline Lambdas keep polling their queues at roughly six requests per minute each.

To address both, the repository ships a `teardown.sh` script that does two things: it disables all four SQS event-source mappings (via `toggle_pipeline.sh`)

so the Lambdas stop polling, and it terminates the Elastic Beanstalk environment so the EC2 instance is released. Everything else (DynamoDB tables, S3 buckets and their contents, Cognito users, Lambda function code, CloudFront, the SQS queues themselves) is left in place, since these services are effectively free at idle and contain user data we want to preserve. Bringing the system back is just another `git tag v*`: the deploy workflow recreates the EB environment, re-points it at the latest application version, and the pipeline-deploy step re-enables the SQS mappings as its final action. The teardown / redeploy cycle therefore takes one command in each direction and never requires touching the AWS console.

5 Twelve-Factor Methodology

The project was designed to follow the twelve-factor methodology as closely as possible within the scope and time constraints of a university project. Applying these principles helped keep the system portable, configurable, and easier to deploy across local and cloud environments.

5.1 Codebase

The project uses a single GitHub repository containing the backend, frontend, infrastructure scripts, documentation, tests, and CI/CD workflows. This made it easier for all team members to work from the same source of truth and review changes consistently.

The repository is organized into clear folders:

- `backend/`: FastAPI application, Lambda handlers, AWS service integrations, tests, and deployment scripts.
- `frontend/`: Next.js application, UI pages, authentication logic, and API client code.
- `docs/`: architecture notes, DynamoDB schema documentation, Cognito documentation, and project draft material.
- `.github/workflows/`: CI and deployment pipelines.

This structure supports the twelve-factor principle of having one codebase tracked in version control, while still allowing several deployable components to be built from it.

5.2 Dependencies

Dependencies are explicitly declared and installed through package managers. The backend uses Python 3.13 with `uv`, and its dependencies are defined in `pyproject.toml` and locked in `uv.lock`. This includes FastAPI, Uvicorn, boto3,

pydantic-settings, python-jose, requests, and testing tools such as pytest and moto.

The frontend uses Node.js and npm. Its dependencies are defined in `package.json` and locked in `package-lock.json`. This includes Next.js, React, TypeScript, ESLint, and Tailwind-related packages.

This approach avoids relying on globally installed packages and improves reproducibility between team members' machines and CI/CD environments.

5.3 Configuration

Configuration is separated from code and provided through environment variables. The backend reads configuration through `pydantic-settings` in `app/config.py`. Important backend variables include:

- `AWS_REGION`
- `DYNAMODB_ENDPOINT_URL`
- `DYNAMODB_TABLE_NAME`
- `S3_BUCKET_NAME`
- `JOB_PROCESSING_QUEUE_URL`
- `BEDROCK_MODEL_ID`
- `COGNITO_USER_POOL_ID`
- `COGNITO_APP_CLIENT_ID`
- `AUTH_BYPASS`

The frontend also uses environment variables, especially:

- `NEXT_PUBLIC_API_URL`
- `NEXT_PUBLIC_AWS_REGION`
- `NEXT_PUBLIC_COGNITO_USER_POOL_ID`
- `NEXT_PUBLIC_COGNITO_APP_CLIENT_ID`

Templates such as `backend/.env.example`, `frontend/.env.example`, and `backend/scripts/pipeline_env.example.sh` document the required configuration for local development without committing secrets to the repository.

5.4 Backing Services

The application treats AWS services as attached resources. The backend does not hard-code infrastructure details directly in the application logic. Instead, services such as DynamoDB, S3, SQS, Bedrock, and Cognito are accessed through configuration values.

This makes the backend more portable. For example, DynamoDB can be configured to use the real AWS table in production, while local development can use DynamoDB Local through `DYNAMODB_ENDPOINT_URL`. Similarly, changing the S3 bucket or SQS queue does not require modifying application logic, only updating the corresponding configuration values.

5.5 Build, Release, Run

The project separates build, release, and run steps using GitHub Actions and deployment scripts. The CI workflow checks the backend with Ruff, validates schemas, runs tests, lints the frontend, type-checks the frontend, and builds the Next.js application.

Deployment is triggered by version tags matching `v*`. The deployment workflow performs several release tasks:

- creates or reuses Cognito resources;
- deploys the FastAPI backend to Elastic Beanstalk;
- packages and deploys Lambda functions;
- builds the frontend;
- syncs the static frontend output to S3;
- invalidates the CloudFront cache.

This provides a clearer separation between code validation, artifact creation, and runtime execution.

5.6 Processes

The backend follows a stateless process model. FastAPI does not store user session state in memory. User identity is provided through Cognito JWT tokens, and persistent application data is stored in DynamoDB or S3.

The asynchronous processing pipeline is also stateless. Lambda functions receive events from S3 or SQS, process one unit of work, write results to DynamoDB, and terminate. This design supports horizontal scalability and reduces coupling between components.

The frontend stores authentication tokens in the browser local storage, but application data is still retrieved from the backend and AWS-backed persistence layers.

5.7 Port Binding

The backend exposes its API through Uvicorn, usually on port `8000` during local development. The frontend runs locally on port `3000` with Next.js. In production, the backend is exposed through Elastic Beanstalk, while the frontend is served as static files through S3 and CloudFront.

This follows the port binding principle for the backend process, while the frontend follows a static hosting model after build.

5.8 Concurrency

The architecture supports concurrency by decomposing the system into independently scalable process types, following the twelve-factor idea of scaling out via the process model rather than threads inside a single application.

The FastAPI backend handles synchronous API requests and runs as a containerized process on Elastic Beanstalk, which can be scaled horizontally by adding more instances behind the load balancer if needed. Document processing is deliberately moved out of the request path into an asynchronous pipeline composed of S3 events, SQS queues, and Lambda functions. Each Lambda function represents its own process type that AWS scales automatically based on queue depth and event volume.

This separation is important because CV and job document processing can be slow due to Textract and Bedrock calls, which would otherwise block API workers and degrade responsiveness. By isolating these tasks behind queues, the API stays responsive, slow work can be retried independently through the SQS redrive policy, and each component can be scaled or tuned without affecting the others.

5.9 Disposability

The backend and Lambda functions are designed to start quickly and shut down safely. Lambda functions process messages independently, and SQS helps prevent message loss when temporary failures occur. Dead-letter queues are also used so failed messages can be inspected instead of silently disappearing.

The project also includes scripts to disable event source mappings and reduce idle costs when the system is not actively being used. This is especially useful to avoid that the AWS resources run unnecessarily.

5.10 Development and Production Parity

The project aims to keep development and production environments similar by using the same backend framework, same frontend framework, same environment variable structure, and same AWS service interfaces.

However, full parity was difficult to achieve. Local development can use DynamoDB Local, but services such as Cognito, Bedrock, S3, Textract, and SQS are mainly tested through real AWS resources. This created some differences

between local and production testing, but it was a practical trade-off because mocking every AWS service would have required significant additional time.

5.11 Logs

Logging is treated as an important operational concern. The FastAPI backend includes request logging middleware that records request method, path, status code, and request duration. Lambda functions also produce logs through CloudWatch.

CloudWatch is used for centralised monitoring, dashboards, alarms, and debugging. This was especially useful for the asynchronous pipeline, where failures may happen outside the direct user request flow.

5.12 Admin Processes

Administrative tasks are implemented as scripts and one-off commands. Examples include:

- creating the DynamoDB table with `scripts/create_table.py`;
- creating the Lambda IAM role;
- deploying the S3, SQS, Lambda, and event notification pipeline;
- setting up Cognito;
- enabling or disabling SQS event source mappings;
- tearing down or pausing costly resources.

These tasks are run in the same project environment as the application and use the same configuration files. This follows the twelve-factor idea that administrative jobs should be executed as part of the same codebase and release context.

5.13 Limitations

The project does not fully satisfy every twelve-factor principle. Infrastructure is automated through scripts and GitHub Actions, but not fully described with a dedicated infrastructure-as-code framework such as Terraform or AWS CDK. Also, local development is not perfectly equivalent to production because some AWS services are difficult to reproduce locally.

Despite these limitations, the main principles of dependency management, environment-based configuration, stateless processes, attached backing services, centralized logging, and automated deployment are clearly reflected in the final architecture.

6 Development Methodology

The team followed an iterative development methodology. Instead of attempting to implement the full system at once, the work was divided into smaller components: architecture design, backend API, frontend interface, authentication, document processing, AWS infrastructure, monitoring, testing, and documentation. This allowed the team to make progress in parallel while still keeping the final application coherent.

6.1 Division of Responsibilities

The responsibilities were divided according to both technical areas and report sections. Some members focused more on frontend development and the user-facing flow, while others focused on backend endpoints, AWS integrations, authentication, infrastructure, observability, and documentation. This division helped reduce overlap and made it easier to assign ownership of specific problems.

The group also worked collectively on the local setup and integration process, especially around environment variables, AWS credentials, Cognito configuration, and making the application runnable in a development environment. This was important because many of the project issues were not isolated coding problems, but integration problems between local configuration, AWS permissions, and deployed resources.

6.2 Communication and Meetings

The team used regular meetings to coordinate progress, discuss technical decisions, and divide remaining work. During these meetings, we reviewed which parts of the system were already working, which components were blocked, and which tasks had to be prioritized before the final deadline.

Because the project included many AWS services, communication was especially important when changes affected shared resources. For example, changes to Cognito, S3 bucket names, queue names, or DynamoDB schema could impact both backend and frontend development. The team therefore had to communicate configuration changes clearly to avoid mismatches between components.

In addition to meetings, asynchronous communication was used to exchange updates, share errors, and discuss fixes. This was useful when debugging cloud issues, since many AWS errors depend on account permissions, region, or resource state.

6.3 Version Control and Collaboration Tools

GitHub was used for version control and collaboration. The repository contains the backend, frontend, documentation, tests, and deployment workflows. Using a shared repository made it possible to track changes, compare implementations, and keep documentation close to the code.

In addition, GitHub Issues was used to track tasks. Issues were assigned to specific team members, which made ownership explicit and helped coordinate work between meetings. This integrated naturally with the rest of the GitHub workflow, since issues could be linked to commits and pull requests.

GitHub Actions was used for continuous integration. On pushes and pull requests to the main branch, the CI workflow runs backend linting, formatting checks, schema validation, backend tests, frontend linting, TypeScript checks, and frontend builds. This helped detect errors before deployment and encouraged the team to keep the codebase in a runnable state.

The project also contains deployment automation through GitHub Actions. Deployments are triggered using version tags, which supports a clearer release process and reduces the need for manual deployment steps.

6.4 Documentation Process

Documentation was created throughout the project, not only at the end. The repository includes documents describing the DynamoDB schema, Cognito implementation, project draft, pipeline testing, and setup instructions. This was useful because the project has many moving parts, and relying only on memory would have made it difficult for all members to reproduce the setup.

The final report was prepared by dividing sections among team members. Each person documented the parts they worked on or understood best, and the report was then combined into a single document. This helped ensure that the report covered both implementation details and higher-level architectural reasoning.

6.5 Development Environments

The project uses several working environments:

- **Local development:** The backend can be run locally with Uvicorn, and the frontend can be run locally with Next.js. Environment variables are loaded from `.env` and `.env.local`. DynamoDB Local can also be used for some local backend testing.
- **AWS development resources:** Some components, such as Cognito, S3, SQS, Textract, Bedrock, and Lambda, are easier to test using real AWS resources. Each team member can configure their own AWS account and credentials for development.
- **Production-like deployment:** The backend is deployed to Elastic Beanstalk, Lambda functions are deployed separately, and the frontend is built as static files and hosted through S3 and CloudFront.

This environment structure gave the team flexibility. Simple API and UI changes could be tested locally, while cloud-specific features could be tested against AWS.

6.6 Testing and Quality Control

Testing was handled at several levels. The backend includes pytest tests for health checks, uploads, DynamoDB behavior, authentication protection, text extraction, profile structuring, and dead-letter queue handling. The tests use moto to mock AWS services where possible, reducing the need for real AWS credentials during automated tests.

The CI workflow also checks code quality with Ruff and validates extraction schemas. On the frontend side, ESLint, TypeScript checking, and production builds are used to catch UI and type errors.

Manual testing was still necessary because the full system depends on multiple AWS services. End-to-end behavior, such as uploading a PDF, triggering S3 events, processing with Lambda, and storing results in DynamoDB, required real integration testing.

6.7 Project Management Approach

The team worked in an incremental way. The first priority was to define a minimum viable version of the application: authentication, CV upload, job input, document processing, skills-gap analysis, and result retrieval. More ambitious features from the first draft, such as fully generated tailored CVs, cover letters, or advanced dashboards, were treated as optional and adjusted according to available time.

This approach helped the team keep the final system realistic. Since the project involved several unfamiliar services, especially Bedrock, Textract, Cognito, and asynchronous AWS pipelines, it was more important to deliver a working end-to-end architecture than to implement every initially imagined feature.

7 Challenges

Throughout the project we ran into a number of challenges. The following subsections describe some of the most instructive ones, what caused them, and how we resolved them.

7.1 Local Development Setup

A central practical challenge was ensuring that every team member could run the application locally and has access to the required AWS services the system depends on. We decided that each member would provision their own resources rather than share a single environment. This kept individual setups isolated, but it also meant that every person had to complete the full deployment setup themselves and follow each step correctly. Small deviations therefore became a source of unexpected errors: if one of us changed a resource name such as an S3 bucket to suit their own account, that change could break things elsewhere unless it was propagated consistently. We addressed this by documenting the required configuration and moved the fragile steps into shared scripts. `.env.example`

files captured the expected environment variables, while dedicated scripts such as `create_lambda_role.sh` and `setup_cognito.sh` provisioned the IAM role and Cognito resources consistently for everyone.

7.2 Structured output from Bedrock

As mentioned in section 4.3, three parts of the application, profile structuring, job structuring, and the skills-gap analysis, depend on the LLM returning JSON that conforms to a fixed schema. The challenge was that Amazon Bedrock, unlike the OpenAI or Anthropic APIs we had used in previous projects, does not expose a native structured-output mode. Our first attempts relied on prompt engineering alone (asking the model to "respond only with JSON matching this schema") which we found not to be reliable enough. We resolved this by leaning on Claude's tool-use feature: the schema is passed to the model as a tool definition and the model is forced to invoke that tool, which guarantees a schema-conformant response and removes the need for any prompt-parsing or repair logic on the application side. The same pattern is reused across all three call sites, with only the schema swapped out. To keep the schemas in sync with the application code, they are generated from the corresponding Pydantic models on every deploy and bundled into the Lambda zip, so the model always validates against the current shape of the Pydantic classes and CI additionally checks that the committed copies do not drift.

7.3 Idle SQS polling burning through the free tier

A few weeks into the project we received an AWS notification that we had used 85% of the SQS free tier (one million requests per month), which was surprising given that the application had only been exercised manually a handful of times. Investigating the CloudWatch metrics on the queues revealed a steady stream of `ReceiveMessage` calls against all four queues, even during long stretches when nobody was using the application and the queues themselves were empty.

The cause is in how Lambda integrates with SQS: when an SQS event-source mapping is enabled, Lambda's poll fleet continuously polls the queue on the function's behalf. Each poll counts as a billable SQS request regardless of whether a message is returned. With four mappings (the profile and job queues plus their two DLQs) running around the clock, the accumulated request count was on track to exceed the free tier, even though the application itself was almost entirely idle.

The fix was to make these mappings part of the teardown procedure described in section 4.7: `toggle_pipeline.sh` can disable and re-enable all four mappings in one call, and `teardown.sh` disables them alongside terminating the Elastic Beanstalk environment. The deploy pipeline re-enables them as its final step, so a normal tag-push deploy always leaves the system in a usable state. After this change the idle SQS request count dropped to zero while the app was not in use.

8 Scope Changes from the Initial Draft

During the implementation phase, several deviations from the initial project scope were made. These changes were primarily driven by technical feasibility considerations, alignment with system design principles, and the desire to avoid redundant functionality already covered by existing components. As a result, certain features were re-evaluated and either simplified or removed from the final implementation, while remaining documented as potential future extensions.

8.1 AI Chat

In the initial proposal, an AI-driven chat feature was planned to enhance the user profile by asking targeted, adaptive questions. The goal of this feature was to collect additional contextual information that may not be present in a traditional CV, thereby improving the completeness of the user profile.

During the design and implementation review, this feature was reconsidered due to its overlap with existing data extraction and processing mechanisms already used elsewhere in the system. Since similar information could be obtained through the resume parsing pipeline and manual profile editing, the chat-based approach was deprioritized in favor of maintaining a more streamlined architecture. The feature remains a potential extension for future iterations of the system.

8.2 Resume and Cover Letter Generation

The original concept included automated generation of customised CVs and cover letters based on specific job postings. This was later replaced with a recommendation-based system that provides actionable feedback on improving an existing resume rather than generating new documents.

This decision was made to ensure better alignment with the design goals of transparency and user control over their data. In particular, generation of cover letters was removed due to concerns regarding authenticity and the difficulty of accurately representing the personality, intentions, and career motivations of the user. The revised approach focuses on supporting users in improving their own materials rather than fully generating application documents on their behalf.

8.3 Job Posting Scraping

Job posting scraping was initially defined as an optional enhancement intended to automatically collect job listings from external sources. This feature would have expanded the system to a broader job discovery platform in addition to an analysis tool.

However, during implementation, challenges related to the consistency of the extracted data and the variability in external job board structures were identified. These issues highlighted the complexity of maintaining reliable scraping logic across multiple sources. Consequently, this feature was excluded from

the final implementation, while the core system retained manual job posting input as a more stable and controlled alternative. The scraping functionality remains a candidate for future development, particularly if integrated with more structured external APIs or integrated way for companies to post their listings directly into the platform.

9 Time Investment

To ensure a transparent breakdown of individual contributions, Table 2 outlines the actual hours allocated by each team member across the primary project phases. These categories are identical to those defined during the initial time-planning phase, as originally presented in the preliminary project draft [1]. To maintain table legibility, the column headers are abbreviated to correspond to the following tasks:

- **D&R:** Design & Research
- **BE:** Backend Development
- **FE:** Frontend Development
- **INF:** Infrastructure & Deployment
- **T&D:** Testing & Debugging
- **M&C:** Meetings & Coordination
- **DOC:** Documentation & Presentation
- **TOT:** Total Hours Accumulated

Name	**D&R**	**BE**	**FE**	**INF**	**T&D**	**M&C**	**DOC**	**TOT**
Max	3	14	0	3	6	2	7	35
Olle	6	12	0	3	5	2	4	32
Benedikt	4	16	1	6	4	2	4	37
Patrik	3	4	9	6	2	2	7	33
Ilyas	7	8	2	5	1	2	7	32
Total	**23**	**54**	**12**	**23**	**18**	**10**	**29**	**169**

Table 2: Project hours broken down by team member and task

9.1 Project Timeline Deviation and Workload Analysis

Comparing the initial estimation with the actual hours reveals a deliberate shift in project priorities. While the cumulative total reached 169 hours, the internal distribution of tasks deviated from our initial plan, specifically regarding backend development, frontend development, and documentation.

As illustrated in Table 3, Frontend Development (FE) was reduced by 8 hours. This reduction was a strategic decision to minimize frontend focus, as our primary objective was to leverage Amazon Web Services (AWS) and integrate cloud tools, which were tasks that demanded more backend resources. Consequently, these hours were reallocated to Backend Development (BE), which exceeded its budget by 9 hours. Additionally, Documentation & Presentation (DOC) required an extra 9 hours over the planned 20 to thoroughly map out the newly implemented AWS architecture and cloud configurations.

In retrospect, our initial planning could have been improved by conducting a more rigorous alignment analysis. Had we recognized earlier that frontend development was less relevant to our core purpose of AWS tool implementation, we could have proactively allocated fewer hours to FE and budgeted more time for backend development and the extensive documentation that cloud architectures inherently require.

Task	**Planned hours**	**Actual hours**	**Diff.**
Design & research (D&R)	20	23	+3
Backend development (BE)	45	54	+9
Frontend development (FE)	20	12	-8
Infrastructure & deployment (INF)	20	23	+3
Testing & debugging (T&D)	15	18	+3
Meetings & coordination (M&C)	10	10	0
Documentation & presentation (DOC)	20	29	+9
Buffer	10	0	-10
Total	**160**	**169**	**+9**

Table 3: Project time estimation: Planned vs. Actual hours with deviations

10 Conclusion

This project successfully implemented a cloud-native Job Application Assistant, directly fulfilling our primary motivation to gain deep, hands-on expertise within the AWS ecosystem. By developing a fully functional pipeline using a Next.js frontend, a FastAPI backend, and an asynchronous serverless processing framework, the team successfully translated theoretical cloud concepts into a robust,

deployed infrastructure.

The execution of the project provided invaluable insights into distributed cloud architectures. A major takeaway was the practical application of key cloud principles such as decoupling and scalability. Working with these managed services allowed us to directly apply and reinforce the foundational understanding of cloud components that was developed during the course laboratories. By taking these concepts from structured lab environments into a complex, end-to-end project, we gained a much deeper appreciation for how decoupled systems prevent systemic failures and maintain reliability under varying workloads.

Looking ahead, several improvements can be made to transition the system into a production-grade application. Future work should focus on strengthening token security and refining user session management, such as implementing global sign-out routines within AWS Cognito. Additionally, integrating features like automated job posting feeds would significantly enhance the platform's overall utility.

References

[1] Max Eidet, Olle Pettersson, Benedikt Prisett, Patrik Timko, and Ilyas Bakchiche. CCBDA Project First Draft. Project Plan, Universitat Politècnica de Catalunya, April 2026.